ConnectomeChronicle

A Temporal, Federated and Distributed Database for the Evolution of Neural Connectomes

---

1. Abstract

Connectomics reconstructs and analyses the structural connectivity of nervous systems. A connectome is naturally a large graph — neurons as nodes, synapses as edges — enriched with cell types, anatomical regions, morphology, neurotransmitter annotations and experimental metadata. Modern datasets are already substantial: the whole-brain female *Drosophila* connectome reported by Dorkenwald et al. contains roughly 139,255 neurons and 54.5 million synapses. As these datasets grow, are repeatedly corrected, and are released in successive versions, the challenge extends beyond biology into data management.

ConnectomeChronicle is a database-first framework for integrating, managing and analysing heterogeneous, evolving connectome datasets. It will ingest sources such as MaleCNS, FlyWire, hemibrain and MANC through source-specific pipelines, transform their schemas into a canonical representation, and retain original identifiers and provenance throughout. An explicit entity-resolution layer maintains cross-dataset correspondences with confidence and evidence. Data is stored using temporal — and, where feasible, bitemporal — representations so historical states can be reconstructed rather than overwritten.

A **ConnectomeDiff** engine compares releases or datasets to identify additions, deletions and modifications in neurons, annotations and connectivity. For scalability, the graph will be distributed across shards using identifier-based, anatomical-region-based and connectivity-community-based partitioning. PostgreSQL is the authoritative relational store; a document-oriented NoSQL database accommodates heterogeneous source metadata. A supporting AI/ML layer uses graph embeddings or GraphSAGE to propose cross-dataset entity matches, and Isolation Forest to flag statistically unusual changes surfaced by the diff engine.

The project's core focus remains **database engineering** — integration, temporal storage, provenance, graph comparison, partitioning and distributed querying.

---

2. Introduction

Advances in electron microscopy, automated segmentation and computational processing now allow neural connectivity to be reconstructed at synaptic resolution. These datasets are graph-structured, but they also carry rich metadata needed for scientific interpretation — so managing a connectome means more than storing a graph; it requires handling large numbers of interlinked entities, heterogeneous metadata, and successive releases.

A further complication is that connectome projects don't share a common representation. One dataset identifies a neuron by `bodyId`, another by `root_id`, another by `neuron_id`; cell type, classification, and region may all be named and structured differently. Combining resources therefore requires understanding which fields correspond conceptually, preserving each source's original identities, and representing uncertain relationships between potentially matching biological entities — not simply copying data into one table.

Connectomes are also temporal by nature. A neuron may start unclassified and later receive a cell-type label; a connection may be revised after proofreading; a dataset may re-release an entire circuit. Overwriting prior records erases the ability to trace how understanding evolved. Infrastructure such as CAVE demonstrates the practical value of version-aware connectome management and point-in-time querying.

ConnectomeChronicle treats a connectome as an **evolving scientific data object**, combining heterogeneous integration, explicit entity mapping, temporal storage, provenance, graph comparison and distributed processing into one experimental framework.

---

3. Problem Statement

The central problem is managing multiple connectome datasets that differ in schema, identifiers, annotations and release history, while still letting researchers compare them consistently and reproducibly.

Consider three datasets where the same conceptual attribute appears as `bodyId`, `root_id`, or `neuron_id`, and cell type appears as `cell_type`, `type`, or `classification`. A direct merge cannot guarantee equivalent records align correctly, and blindly substituting one identifier for another destroys traceability to the original source. The problem deepens when two datasets contain entities that may correspond biologically but differ in identity and annotation certainty.

The architecture must therefore:

- Preserve source-specific records and identities **while** building a common canonical representation
- Explicitly represent entity correspondences, with evidence and confidence, rather than hide them in preprocessing code
- Preserve historical record states across releases
- Support determining what changed, where it came from, and how the resulting graph can be queried efficiently when distributed across nodes

Research question:

> *How can heterogeneous and evolving neural connectome datasets be integrated into a provenance-aware temporal database while preserving source identity and enabling efficient comparison and distributed querying of their graph structures?*

---

4. Motivation and Significance

Recent whole-brain datasets contain hundreds of thousands of entities and tens of millions of synaptic relationships, making organization and retrieval a genuine computational problem. Large-scale graph analyses of FlyWire show that researchers increasingly need to treat connectomes as weighted, directed networks rather than collections of independent records.

Connectome data is also never static — proofreading, annotation and new releases continually change it. CAVE demonstrates that temporal access and reproducibility are practically valuable. Cross-connectome work such as the FlyWire–hemibrain study shows that comparing entities across resources is itself complex, involving morphology, connectivity and evidence-based correspondence.

These trends motivate an architecture where integration and historical evolution are first-class data-management concerns — not a system built to visualize a single connectome, but one that studies how multiple datasets can be **combined without losing provenance, maintained over time, compared systematically, and distributed for scalable querying**.

---

5. Literature Survey

5.1 Existing Connectome Infrastructure

- **FlyBrainLab / NeuroArch** integrate diverse *Drosophila* neurobiological data in a common environment, showing the value of a shared representation for heterogeneous scientific data.
- **neuPrint** offers graph-oriented storage and querying for connectomic datasets, demonstrating the practicality of graph databases at this scale.
- **Dorkenwald et al. (2024)** provide a whole-brain *Drosophila* connectome, illustrating the scale of modern connectomic data generation.
- **Lin et al. (2024)** analyse FlyWire as a large directed, weighted graph, reinforcing the value of graph-based computation.
- **CAVE** is especially relevant to the temporal component: it supports proofreading, annotation and point-in-time querying, meaning ConnectomeChronicle cannot claim temporal management as wholly novel — instead, it combines temporal representation with integration, provenance, diffing and distributed experimentation.
- **Schlegel et al. (2024)** developed systematic annotation and comparative cell typing across FlyWire and hemibrain, showing that cross-connectome correspondence is not a trivial identifier lookup — motivating explicit, evidence-bearing mappings.
- **NTAC** and **NeuroGraphBench** (2026) show that connectivity-based neuron classification and AI-agent connectome exploration are already active research areas — so ConnectomeChronicle deliberately keeps AI as a supporting role (entity-matching assistance and anomaly detection) rather than a primary contribution.

5.2 Comparison of Existing Systems

System / Work	Year	Main Contribution	Relevance	Remaining Gap
FlyBrainLab / NeuroArch	2021	Integrates diverse *Drosophila* neurobiological data via a unified computational environment	Establishes the need for heterogeneous integration and a common representation	No focus on historical state, explicit provenance, diffing, or distributed partitioning
neuPrint	2022			

System / Work	Year	Main Contribution	Relevance	Remaining Gap
		Graph-oriented connectome storage, APIs and querying	Motivates graph-oriented data management and querying	No cross-dataset integration, temporal evolution, or distributed DB experimentation
Dorkenwald et al.	2024	Whole-brain *Drosophila* connectome and associated resources	Establishes scale and complexity of modern connectome data	Not a general temporal, provenance-aware, distributed integration framework
Schlegel et al.	2024	Whole-brain annotation and multi-connectome cell typing (FlyWire × hemibrain)	Directly motivates cross-dataset entity resolution	Correspondence studied as biological analysis, not as persistent DB state with provenance
Lin et al.	2024	Large-scale graph analysis of FlyWire	Motivates graph storage and large-scale processing	No temporal/differential DB management or distributed storage experiments
CAVE	2025	Proofreading, annotation and historical querying	Establishes the importance of temporal connectome data	Not the same combined integration + diff + distributed framework
NTAC	2026	Connectivity-based ML for neuronal type assignment	Supports feasibility of graph-based ML in connectomics	Neuron classification is not ConnectomeChronicle's primary ML task
NeuroGraphBench	2026	AI-agent-based exploration of *Drosophila* connectomes	Shows growing use of AI in connectome analysis	Project focuses on DB infrastructure, not AI-agent interaction

6. Identified Research Gap

Individually, the building blocks of ConnectomeChronicle are established: heterogeneous integration (NeuroArch, FlyBrainLab), graph-oriented querying (neuPrint), temporal management (CAVE), and cross-connectome correspondence (FlyWire/hemibrain research). The gap is their **integration into a single, experimentally evaluated DBMS architecture** — one that preserves source identity alongside canonical entities, retains confidence and provenance for mappings, reconstructs and compares versions, and distributes the resulting graph across shards under different partitioning strategies. The AI/ML layer adds targeted intelligence without displacing the database as the system of record.

7. Objectives

7.1 Primary Objectives

1. Design a unified database architecture that integrates heterogeneous connectome datasets with different schemas, identifiers and annotations — retaining source identities while enabling consistent cross-dataset querying and comparison.
2. Implement temporal, version-aware storage so the evolution of neurons, connections and annotations is preserved rather than overwritten; investigate a bitemporal model to distinguish scientific validity time from database recording time.
3. Develop a provenance-aware **ConnectomeDiff** framework to compare releases or datasets and identify added, deleted and modified neurons, annotations and connections, with full source and transformation history.
4. Investigate distributed storage strategies — identifier-based, anatomical-region-based and connectivity-community-based partitioning — and study their effect on query latency, cross-shard communication, storage balance and join overhead.
5. Experimentally evaluate the architecture for storage efficiency, historical reconstruction, graph-diff computation, distributed query performance and scalability.

7.2 Secondary Objectives

1. Introduce a graph-based ML component (graph embeddings and/or GraphSAGE) to generate candidate cross-dataset entity matches; final mappings remain database-controlled, stored with confidence and evidence.
2. Implement an Isolation Forest anomaly-detection layer over ConnectomeDiff output to flag statistically unusual connectivity or annotation changes for review — not to assert biological error.
3. Provide an API and dashboard for querying integrated data, reconstructing historical states, inspecting cross-dataset mappings, comparing versions, tracing provenance and reviewing anomalies.

---

8. Proposed System — Overview

Public connectome datasets pass through source-specific ingestion pipelines into a staging layer. Schema normalization maps source fields to a canonical model while retaining source-specific identifiers and metadata. Entity resolution then determines possible relationships between source records and canonical entities. The result is stored in relational and document-oriented databases, with temporal and provenance information attached throughout.

The integrated database can reconstruct historical states and compare them via ConnectomeDiff. The large graph can be partitioned across multiple database instances and queried through a routing layer, with ML operating on top of — not in place of — these database functions.

---

9. Data Integration and Canonicalization

9.1 Source-Preserving Ingestion — Each dataset is first imported in its original form, so identifiers and metadata remain available for provenance and later verification (e.g., `bodyId = 172` from MaleCNS and `root_id = 8392` from FlyWire stay distinguishable even after mapping to a common representation).

9.2 Schema Harmonization — A schema-mapping registry defines how source fields correspond to canonical concepts (e.g., `bodyId`, `root_id`, `neuron_id` → a source-identity field; `cell_type`, `type`, `classification` → levels of a canonical annotation hierarchy, where meanings permit). This is semantic and structural harmonization, not simple column renaming.

9.3 Canonical Representation — Normalized data is represented through entities such as `CanonicalNeuron`, `NeuronType`, `BrainRegion`, `Connection` and `DatasetRelease`. The canonical identifier is an internal system identity that does not replace source-specific identity:

```
Source Dataset
      |
      +---- Source Entity
                  |
                  | mapping
                  v
            Canonical Entity
```

This lets one canonical entity carry multiple source representations without losing original information.

9.4 Entity Resolution — Determines whether source-specific entities correspond to each other or to an existing canonical entity. Each mapping record stores the participating datasets and identifiers, plus matching method, confidence and evidence. Because cross-connectome correspondence can be ambiguous, the database permits uncertain or unresolved mappings rather than forcing every source entity into a single canonical identity.

---

10. Graph-Based ML for Entity Resolution

Once datasets are normalized, the graph structure around a neuron — upstream/downstream neighbours, weighted connections, local topology — can generate candidate cross-dataset matches. A graph embedding method converts this into a numerical representation that can be compared across datasets; GraphSAGE is investigated as an advanced approach, with a simpler embedding method as baseline.

Example candidate ranking:

```
FlyWire neuron 8392

MaleCNS neuron 172    similarity = 0.94
MaleCNS neuron 193    similarity = 0.72
MaleCNS neuron 221    similarity = 0.61
```

The ML component produces **candidates**, not final identities — these can be combined with cell-type similarity, anatomical-region agreement, and existing published mappings. The selected or reviewed correspondence is then stored with confidence and provenance, keeping the database authoritative.

---

11. Temporal and Bitemporal Storage

Dataset releases are represented explicitly, each with an identity, version and release date; historical records for neurons, connections and annotations link to the relevant release rather than being overwritten. Example neuron history:

```
v0.8 → Unknown
v0.9 → LC12 candidate
v1.0 → LC12
```

Where the data supports it, the system distinguishes **valid time** (when a biological statement is considered applicable) from **transaction time** (when the database recorded it) — enabling questions about both scientific validity and the history of database knowledge.

---

12. Provenance Management

Every canonical fact should be traceable to its source dataset, source release, original record, and any transformation or mapping used to derive it:

```
Canonical Fact
      |
Canonical Entity
      |
Source Entity
      |
Dataset Release
      |
Original Record
      |
Transformation
```

This matters most when ML produces a candidate mapping or ConnectomeDiff reports a change — researchers must be able to distinguish information that comes directly from a dataset from information generated by a transformation or model.

---

13. ConnectomeDiff

ConnectomeDiff compares two historical states or datasets, after canonical mappings are established, identifying structural and attribute-level differences — new entities, removed entities, and changes to annotations or connectivity.

Example:

```
Version 0.9              Version 1.0

A → B : 31                A → B : 39
B → C : 14                B → C : 14
C → D : --                C → D : 8
```

Resulting diff:

```
A → B      +8 synapses
B → C       Unchanged
C → D       Added
```

Each difference record links to its provenance and can be analysed by the AI/ML anomaly-detection layer.

---

14. AIML-Based Change Analysis

Isolation Forest analyses the change records ConnectomeDiff produces — it does not independently inspect raw biological data for correctness; the database first establishes the exact changes and a structured feature representation.

Possible features: percentage change in connection weight, change in node degree, number of added/deleted connections, change in local connectivity, historical variance, annotation changes.

A change from 9 to 52 synapses, for instance, would likely receive a much higher anomaly score than a change from 9 to 11 synapses. Results are presented as **an unusual change worth review**, not as proof of a biological error.

---

15. Distributed and Federated Database Architecture

ConnectomeChronicle will deploy multiple PostgreSQL instances/containers acting as shards. In the student prototype, this emulates a federated/distributed environment rather than a true multi-institution federation of independently administered databases. A query router determines which shard(s) a query needs, enabling study of how partitioning affects distributed query behaviour:

```
                  REST API
                     |
                Query Router
              /      |      \
             /       |       \
PostgreSQL 1 PostgreSQL 2 PostgreSQL 3
   Shard 1      Shard 2      Shard 3
```

---

16. Graph Partitioning Strategies

Strategy	Approach	Purpose
Identifier-based	Neuron IDs divided into ranges or hash-based groups	Simple baseline for measuring distributed behaviour
Anatomical	Neurons grouped by region (optic lobe, central brain, ventral nerve cord)	Improve locality for region-specific queries
Connectivity-community	Graph clustering groups strongly interconnected neurons, then assigns groups to shards	Reduce edges crossing shard boundaries

---

17. SQL and NoSQL Database Design

PostgreSQL is the authoritative relational database, since most core concepts (canonical neurons, releases, mappings, temporal records, connections, provenance, partition metadata) involve strong relationships and integrity constraints best served by primary/foreign keys, indexes and transactions.

A document-oriented database (e.g., MongoDB) complements PostgreSQL by holding heterogeneous source-specific information that varies between datasets — source-specific annotations, morphology fields, semi-structured experimental metadata. It does not duplicate relational state; it addresses the schema variability that motivates integration in the first place.

Store	Responsibility
PostgreSQL	Structured, authoritative state; temporal records; relationships; provenance; mappings; transactions
MongoDB	Heterogeneous source metadata; flexible annotations; dataset-specific structures; semi-structured evidence

---

18. Functional Requirements

- Ingest selected connectome datasets through modular, source-specific pipelines, preserving original identifiers and records.

- Normalize heterogeneous schemas into a canonical representation while retaining the source-to-canonical field mapping.
- Support explicit entity resolution between source records and canonical entities, retaining identifiers, confidence, mapping method and evidence — including unresolved or uncertain mappings.
- Provide release-aware, temporal storage of neurons, connections and annotations; allow reconstruction of historical states and comparison of two releases or datasets without manual rebuilding.
- Maintain provenance for integrated records and generated mappings, with traceability back to source release and information.
- Run ConnectomeDiff to identify added, deleted and modified entities/connections, and support Isolation Forest-based anomaly analysis of those changes.
- Route queries across multiple database shards and support evaluation of alternative partitioning strategies.
- Provide an API and dashboard for querying integrated data, inspecting historical states, comparing releases, examining provenance and reviewing ML-generated results.

---

19. Non-Functional Requirements

- **Performance:** common metadata and historical-state queries should target roughly two seconds or less under the prototype's experimental environment; actual figures will be reported alongside dataset size and hardware configuration.
- **Integrity:** transactions, constraints and referential relationships must ensure failed ingestion operations do not corrupt previously committed state.
- **Reproducibility:** historical and provenance information must remain reproducible; comparison/ML outputs should be tied to the dataset versions, feature configuration and model version used to produce them.
- **Modularity:** new connectome datasets should be addable via new adapters without redesigning the canonical model.
- **Security:** database credentials and administrative operations must be protected; the dashboard should expose only data and functionality appropriate to the user's role.

---

20. Software and Hardware Requirements

Layer	Tooling
Backend / API	Python, FastAPI
Relational storage	PostgreSQL
Document storage	MongoDB
Data handling	Pandas, NumPy
Graph processing	NetworkX (or equivalent)

Layer	Tooling
Classical ML	Scikit-learn (Isolation Forest)
Graph ML	PyTorch Geometric (GraphSAGE)
Deployment	Docker, Docker Compose
Frontend	React, Vite

Hardware: a modern multi-core PC with ~16 GB RAM or more and sufficient storage for the chosen datasets should suffice for a prototype. A GPU benefits graph-learning experiments but is not mandatory for the core database implementation.

---

21. System Architecture



The AI/ML layer sits logically above the database: graph embeddings assist entity-resolution candidates, and Isolation Forest analyses database-produced changes. Neither model replaces the canonical store, temporal reconstruction mechanism, or provenance system.

---

22. Experimental Methodology

1. **Snapshot vs. Delta Storage** — Compare storing every release as a full snapshot against storing an initial state plus deltas. Measure total storage, update cost and historical-reconstruction time.
2. **Distributed Partitioning** — Compare identifier-based, anatomical and connectivity-community partitioning. Measure query latency, cross-shard edges, network transfer, distributed joins and load imbalance.
3. **Integrated vs. Independent Data Access** — Compare querying multiple datasets independently against querying the canonical integrated representation, in terms of query complexity, execution time and transformation count.
4. **Entity Resolution** — Compare a rule-based baseline against graph-embedding-based candidate generation and, where feasible, GraphSAGE. Report precision, recall, F1, mapping coverage and inference time where reference mappings exist.
5. **Change Anomaly Detection** — Compare Isolation Forest against a simple threshold-based detector, reporting precision/recall/F1/false-positive rate where labelled data exists, or exploratory anomaly ranking otherwise.

---

23. Expected Outcomes

- A functional prototype integrating heterogeneous connectome datasets while retaining source-specific information and canonical entities, letting researchers explore cross-dataset correspondences without losing original identities.
- Release-aware historical reconstruction, provenance tracing and version comparison, so researchers can determine how a neuron, annotation or connection changed and trace that change to its source.
- Experimental evidence on how partitioning strategies affect storage locality, cross-shard communication and query latency.
- Demonstrated AI/ML support: graph-based candidate mapping generation and statistically-informed anomaly detection — enhancing usability without changing the database's role as system of record.

---

24. Scope and Limitations

The project integrates a **selected** set of publicly accessible connectome datasets and releases, not every available resource, and concentrates on database representation, integration, temporal management, provenance, comparison and distributed querying. The AI/ML component is limited to graph-based candidate matching and anomaly detection.

The project will **not**: generate new biological connectomes, provide clinical or diagnostic conclusions, or automatically declare a connectome change biologically erroneous. It does not aim to reproduce the full functionality of established platforms such as FlyWire, neuPrint, NeuroArch or CAVE — its purpose is a distinct database-oriented combination of capabilities.

Entity-resolution evaluation quality depends on the availability of reliable cross-dataset reference mappings, and the distributed implementation initially runs on local Dockerized instances — a prototype of distributed/federated behaviour rather than a production-scale, multi-institution federation.

---

25. Conclusion

ConnectomeChronicle addresses the database-management challenges created by the growing scale, heterogeneity and evolution of modern neural connectome datasets. Existing systems already establish important capabilities in integration, graph querying, temporal annotation management and cross-connectome analysis — so this project does not claim to replace them or that any single capability is novel.

Instead, ConnectomeChronicle investigates how these concerns come together in a **single DBMS-oriented architecture**: preserving source identities alongside canonical entities, explicitly storing uncertain cross-dataset mappings, maintaining historical and provenance-aware state, providing ConnectomeDiff for graph evolution, and evaluating distributed storage strategies for large connectome graphs. PostgreSQL and NoSQL storage address different forms of heterogeneity, while the distributed layer provides an experimental environment for studying graph-oriented database performance.

The AI/ML layer strengthens the system without changing its identity — graph embeddings and GraphSAGE assist candidate entity correspondence, while Isolation Forest identifies statistically unusual changes after the database has reconstructed and compared historical states. The result treats the connectome not as a static dataset, but as a **heterogeneous, evolving, provenance-aware graph** whose history and structure can be stored, queried, compared and experimentally distributed.

---

26. References

- Dorkenwald, S. et al. (2024). *Neuronal Wiring Diagram of an Adult Brain.* — Provides the large-scale whole-brain *Drosophila* connectome motivating the storage and graph-processing requirements.
- Schlegel, P. et al. (2024). *Whole-Brain Annotation and Multi-Connectome Cell Typing of Drosophila*. — Primary evidence motivating the cross-connectome entity-resolution layer.
- Lin, A. et al. (2024). *Network Statistics of the Whole-Brain Connectome of Drosophila*. — Motivates graph-oriented processing of large connectome networks.
- CAVE — *Connectome Annotation Versioning Engine* (2025). — Precedent for temporal, version-aware connectome management and reproducible access.
- FlyBrainLab / NeuroArch (2021). — Precedent for integrating heterogeneous *Drosophila* neurobiological data through a unified representation.
- neuPrint (2022). — Precedent for graph-oriented storage and querying via Neo4j and supporting APIs.
- NTAC — *Neuronal Type Assignment from Connectivity* (2026). — Context for using graph ML selectively rather than as the project's primary contribution.
- NeuroGraphBench (2026). — Demonstrates AI-agent-based connectome exploration, motivating keeping LLM-based interaction outside the project's central scope.